

**LAPORAN PRAKTIKUM
TEKNOLOGI IoT II**

JARINGAN NIRKABEL DAN KOMUNIKASI HTTP



Disusun Oleh:

Salma Anggun Velyza (25782082)

TRI 3C

Dosen Pengajar:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

2025/2026

Modul 3: Jaringan Nirkabel dan Komunikasi HTTP

Kebutuhan Perangkat Keras

Pastikan komponen berikut tersedia di meja Anda:

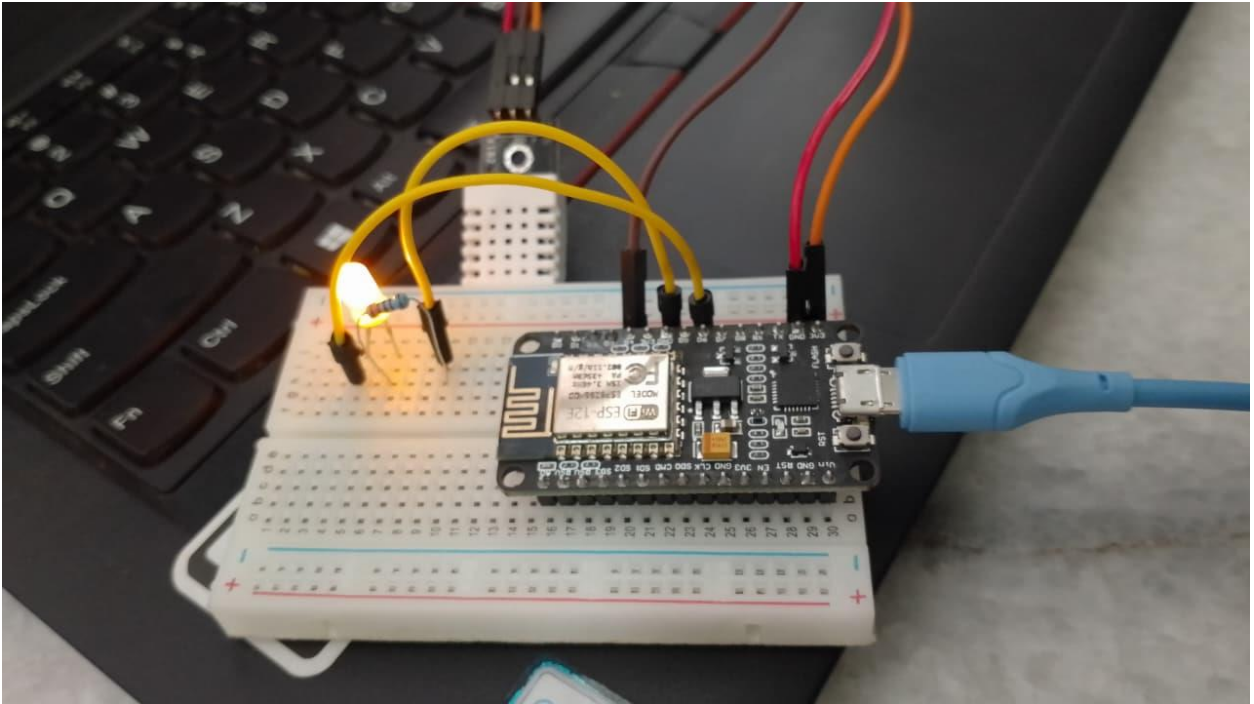
- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- Sensor Suhu & Kelembapan (DHT11 atau DHT22) (1 buah)
- Sensor LDR (1 buah)
- Resistor 10k Ohm (1 buah, untuk pembagi tegangan LDR)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah, untuk pembatas arus LED)
- Kabel *Jumper Male-to-Male* dan *Male-to-Female* (secukupnya)

Persiapan Perangkat Lunak & Jaringan

1. Pada Praktikum Bab ini, seluruh NodeMCU membutuhkan koneksi ke jaringan lokal Wi-Fi yang sama (satu *subnet*).
2. Sangat disarankan Anda menggunakan fitur *Tethering/Mobile Hotspot* dari ponsel Anda untuk memperlancar praktikum, mengingat jaringan Wi-Fi kampus seringkali menerapkan isolasi klien (*AP Isolation*) yang memblokir komunikasi antar NodeMCU.
3. Pastikan Anda mencatat nama **SSID** dan **Password** dari *Hotspot* Anda dengan akurat.

PRAKTIKUM 1: NODEMCU SEBAGAI *WEB SERVER* (MENYAJIKAN HTML)

- The screenshot displays the Arduino IDE interface. The main editor window shows a C++ sketch for a NodeMCU 1.0 (ESP-12E) module. The code includes headers for ESP8266WiFi, ESP8266WebServer, and DHT. It defines a server on port 80 and a DHT sensor on pin 12. A simple web page is served when the root URL is accessed, displaying the sensor's temperature. The Serial Monitor at the bottom shows the output of the program, which is the IP address of the server (10.241.101.146) repeated four times. The status bar at the bottom indicates the board is 'NodeMCU 1.0 (ESP-12E Module) on COM4' and the temperature is 33°C.



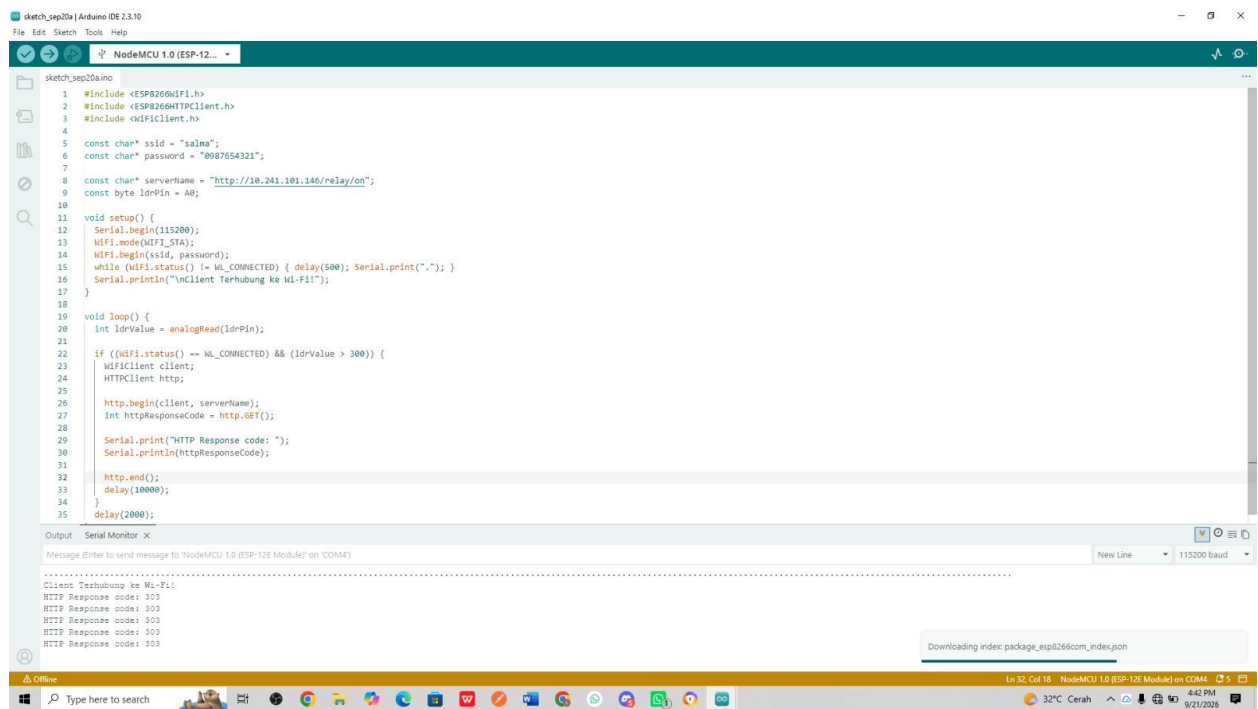
Penjelasan Singkat Kode:

- `ESP8266WebServer server(80);`: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- `html.replace("%TEMPERATURE%", String(t));`: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder %TEMPERATURE%*, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- `server.on("/relay/on", handleRelayOn);`: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran `/relay/on`, server akan menjalankan aksi mengubah *state* LED (menyalakan/mematikan output), lalu *me-redirect* layar pengunjung kembali ke halaman depan (*Location: /*) via kode 303.
- `server.handleClient();`: Berada di dalam `loop()` untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

PRAKTIKUM 2: NODEMCU SEBAGAI *HTTP CLIENT* (MEMINTA LAYANAN)

Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.
2. Buat berkas baru (*New Sketch*).
3. Tuliskan kode program di bawah. Ganti variabel `serverName` dengan IP Address NodeMCU Praktikum 1 milik teman sebelah Anda. (Pastikan Anda menggunakan Wi-Fi yang sama persis).
4. Upload kode tersebut, dan buka Serial Monitor. Saat LDR Anda di *Client* ditutup tangan hingga gelap, perhatikan LED di NodeMCU *Server* milik teman Anda pasti akan menyala!



```
sketch_sep03a | Arduino IDE 2.3.10
File Edit Sketch Tools Help

sketch_sep03a.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "salma";
6 const char* password = "8987654321";
7
8 const char* serverName = "http://10.241.101.146/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   int ldrValue = analogRead(ldrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23     WiFiClient client;
24     HTTPClient http;
25
26     http.begin(client, serverName);
27     int httpStatusCode = http.GET();
28
29     Serial.print("HTTP Response code: ");
30     Serial.println(httpStatusCode);
31
32     http.end();
33     delay(10000);
34   }
35   delay(2000);
36 }
```

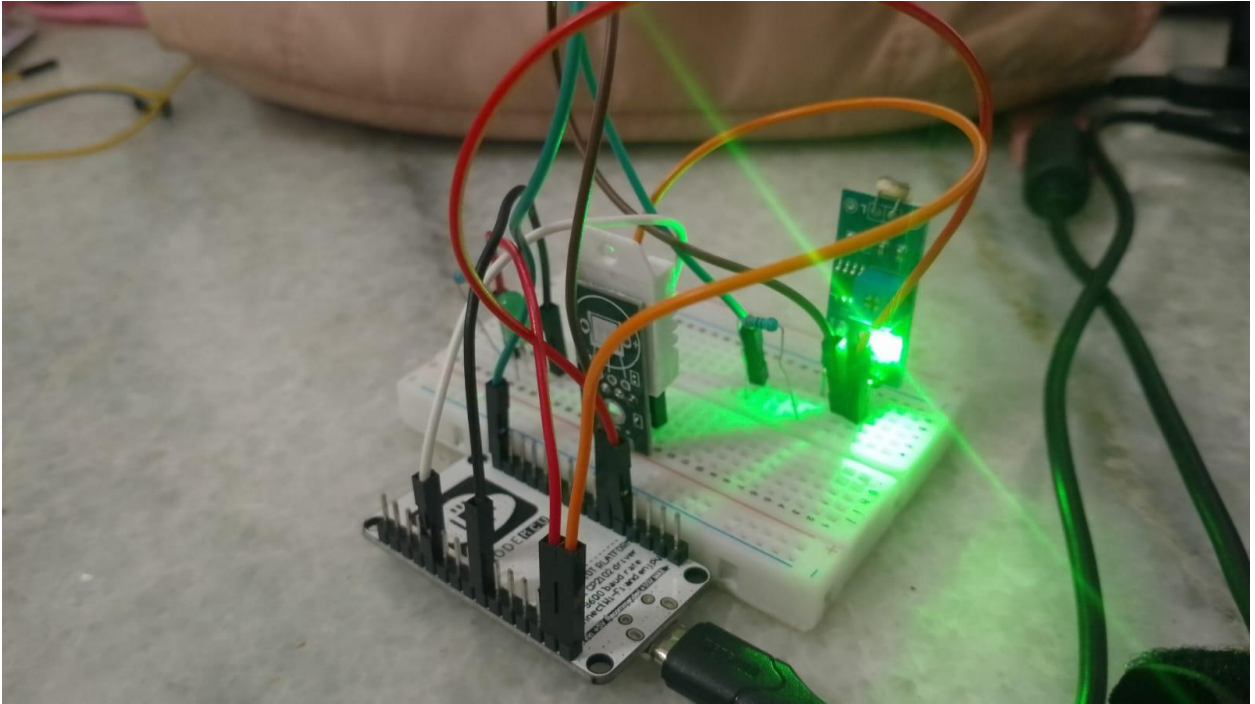
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

Client Terhubung ke Wi-Fi!
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303

Downloading index package_esp8266com_index.json

Ln 32, Col 18 NodeMCU 1.0 (ESP-12E Module) on COM4 4:42 PM 9/21/2026



Penjelasan Singkat Kode:

- `WiFiClient client;` dan `HTTPClient http;`: Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- `http.begin(client, serverName);`: Menargetkan sasaran URL *server*.
- `int httpResponseCode = http.GET();`: Mengeksekusi ketukan pintu (*Request HTTP GET*). Jika *Server* merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- `delay(10000);`: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) *Server* dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

LATIHAN

LATIHAN 1

Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

- ⇒ **Terhubung ke Infrastruktur Jaringan yang Ada:** Dalam mode STA, perangkat NodeMCU/ESP8266 bertindak sebagai klien yang bergabung ke dalam jaringan Wi-Fi lokal yang sudah ada (seperti router rumah atau *hotspot* HP). Hal ini memungkinkan perangkat lain (seperti laptop atau *smartphone*) yang berada di jaringan lokal yang sama untuk mengakses alamat IP server dengan mudah.
- ⇒ **Kemudahan Akses dan Fleksibilitas Pengguna:** Jika menggunakan mode *Access Point (AP)*, NodeMCU akan memancarkan sinyal Wi-Fi sendiri, sehingga pengguna harus memutuskan koneksi internet utama mereka untuk terhubung ke Wi-Fi NodeMCU tersebut. Dengan menggunakan mode **STA**, perangkat pengguna dapat tetap terhubung ke jaringan internet secara normal sambil mengakses serta mengontrol *dashboard* IoT pada alamat IP lokal NodeMCU secara bersamaan.

LATIHAN 2 (Modifikasi Antarmuka & Telemetry):

Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

- i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.
- ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi **LED** saat ini:
 - Jika LED sedang **OFF**, tombol menampilkan tulisan "**NYALAKAN (ON)**" (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.
 - Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan "**MATIKAN (OFF)**" (warna merah) dan mengarahkan aksi ke `/relay/off`.

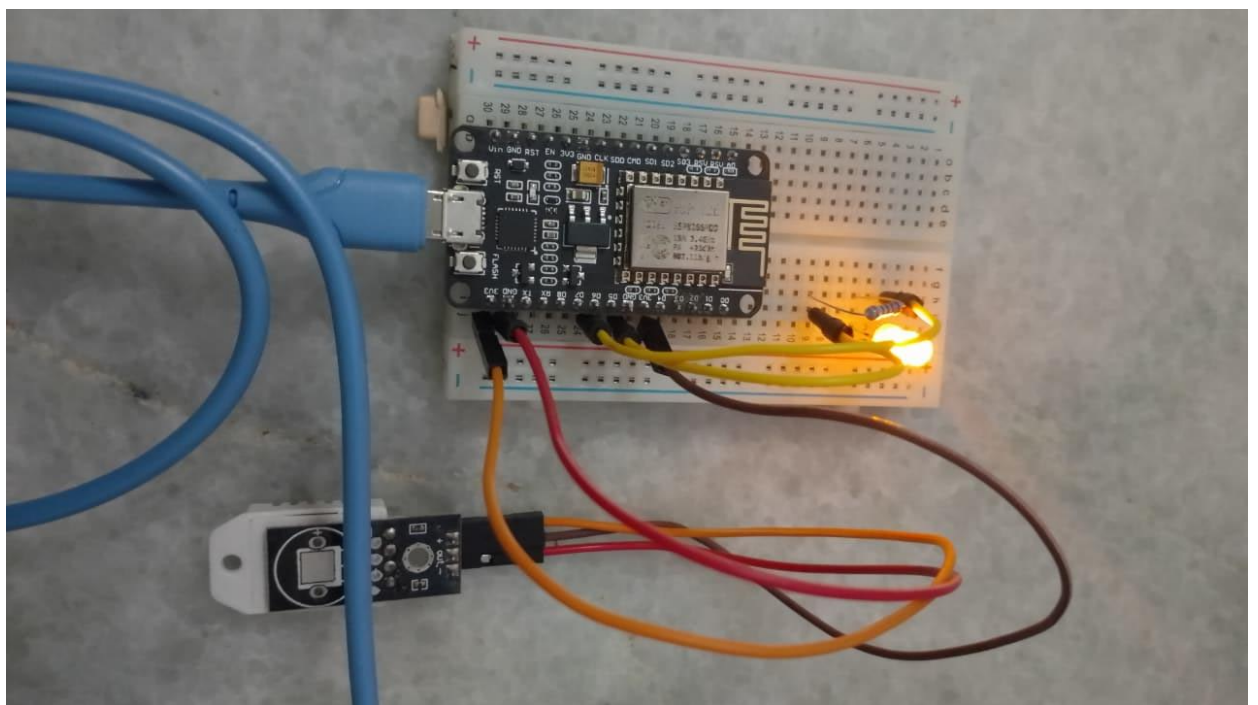
The screenshot displays the Arduino IDE interface with the following components:

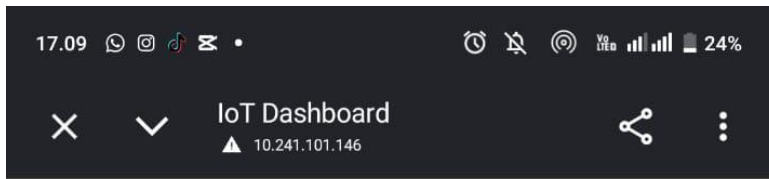
- File Menu:** File, Edit, Sketch, Tools, Help.
- Toolbar:** Includes icons for saving, opening, and running the sketch.
- Sketch Name:** NodeMCU 1.0 (ESP-12E...).
- Code Editor:** Contains the sketch Sep20b.ino with the following code:

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "salma";
6 const char* password = "0987654321";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18 <meta name="viewport" content="width=device-width, initial-scale=1">
19 <title>IoT Dashboard</title>
20 <style>
21   body { font-family: Arial; text-align: center; margin-top: 50px; }
22   button { padding: 15px 30px; font-size: 28px; border-radius: 8px; margin: 18px; cursor: pointer; }
23   .btn-on { background-color: #4CAF50; color: white; border: none; }
24   .btn-off { background-color: #F44336; color: white; border: none; }
25   .sensor-box { font-size: 24px; font-weight: bold; }
26 </style>
27 </head>
28 <body>
```
- Serial Monitor:** Shows the output of the sketch, including HTTP responses and sensor data.

```
Serial Monitor X Output
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line 115200 baud

HTTP Response code: 303
...
Wi-Fi Teshubung!
Alamat IP Web Server: 10.241.101.202
HTTP Web Server telah aktif.
...
Wi-Fi Teshubung!
Alamat IP Web Server: 10.241.101.202
HTTP Web Server telah aktif.
....
IP Address Server Anda:
10.241.101.146
...
IP Address Server Anda:
10.241.101.146
```
- Status Bar:** Shows the current board (NodeMCU 1.0 (ESP-12E Module) on COM4), temperature (30°C), and time (5:10 PM, 9/21/2025).





ESP8266 Web Server

Suhu Saat Ini: **31.40 Celcius**

Kelembapan: **48.90 %**

Kendali Relay

NYALAKAN (ON)

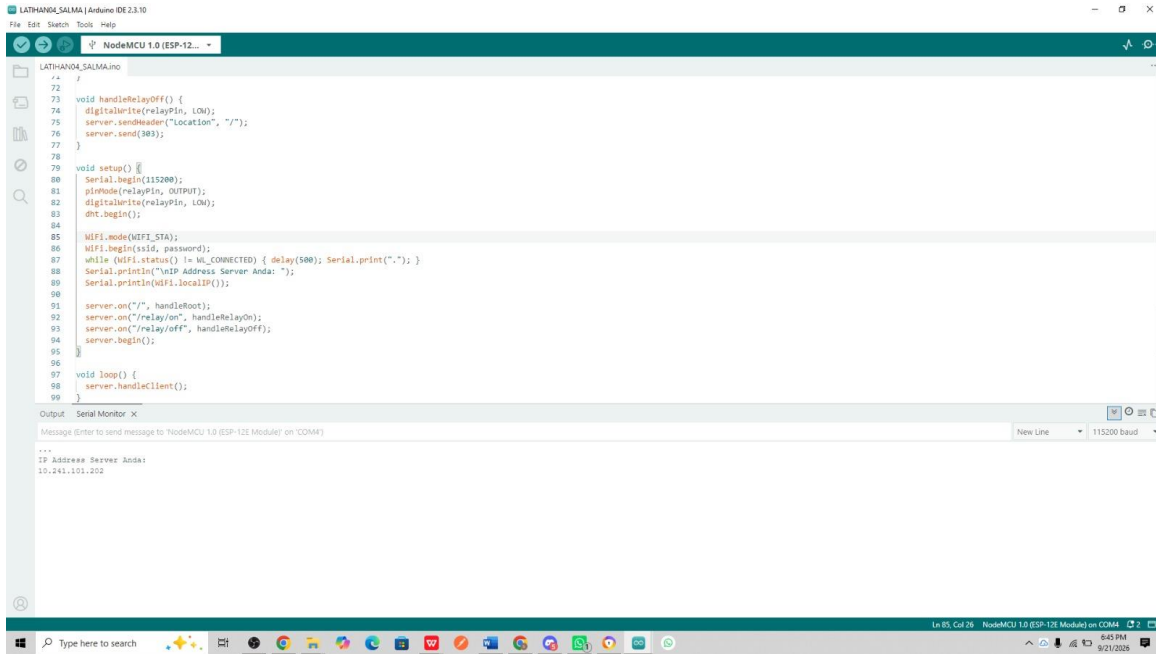
LATIHAN 3 (Analisis Kode Client):

Jika NodeMCU *Server* milik teman mendadak kehilangan daya (*offline*) atau tidak dapat dijangkau, maka *Serial Monitor* pada NodeMCU *Client* tidak akan mencetak kode sukses (seperti 303), melainkan akan mencetak kode status negatif (kode error HTTP Client), umumnya berupa angka negatif seperti:

- HTTP Response code: -1 (atau kode error koneksi lainnya dari library HTTPClient).

LATIHAN 4 (Pengembangan Algoritma):

Memodifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

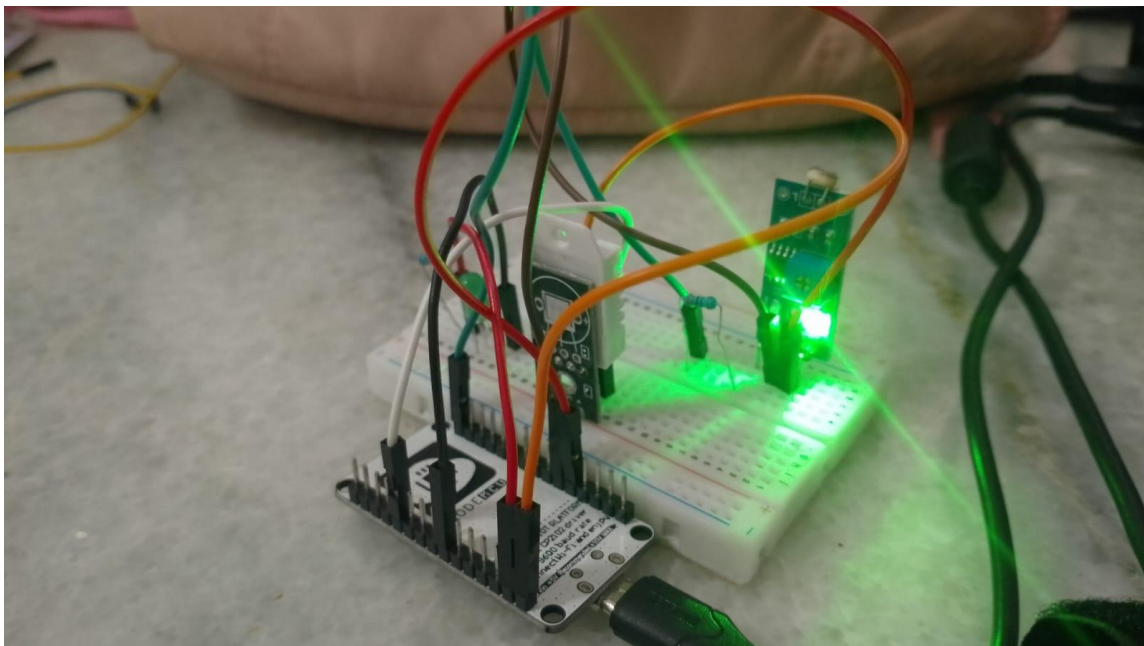


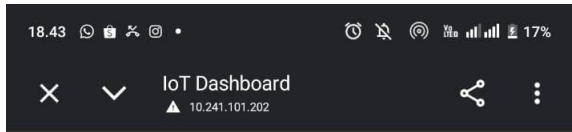
```
1 //
2
3 void handleRelayOff() {
4   digitalWrite(relayPin, LOW);
5   server.sendHeader("Location", "/");
6   server.send(303);
7 }
8
9 void setup() {
10  Serial.begin(115200);
11  pinMode(relayPin, OUTPUT);
12  digitalWrite(relayPin, LOW);
13  dht.begin();
14
15  WiFi.mode(WIFI_STA);
16  WiFi.begin(ssid, password);
17  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
18  Serial.println("WiFi Address Server Anda: ");
19  Serial.println(WiFi.localIP());
20
21  server.on("/", handleRoot);
22  server.on("/relay/on", handleRelayOn);
23  server.on("/relay/off", handleRelayOff);
24  server.begin();
25
26  void loop() {
27    server.handleClient();
28  }
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

...
IP Address Server Anda:
10.241.101.202





ESP8266 Web Server

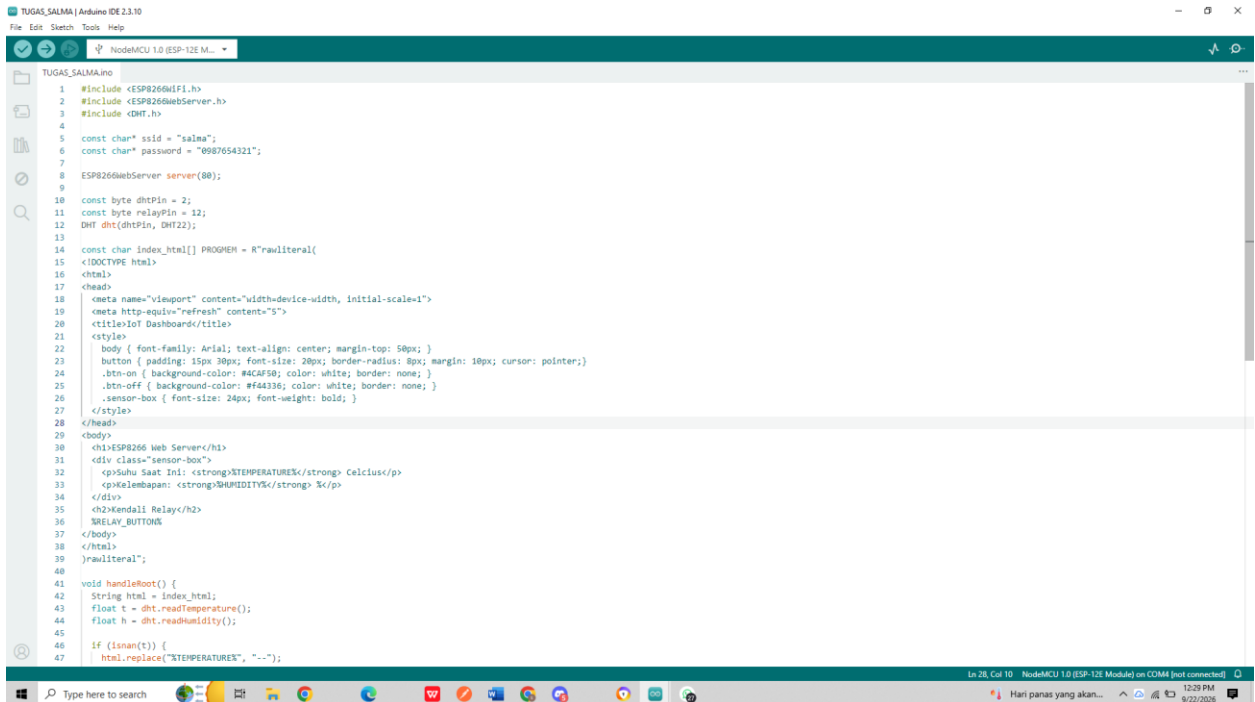
Suhu Saat Ini: -- Celcius

Kelembapan: -- %

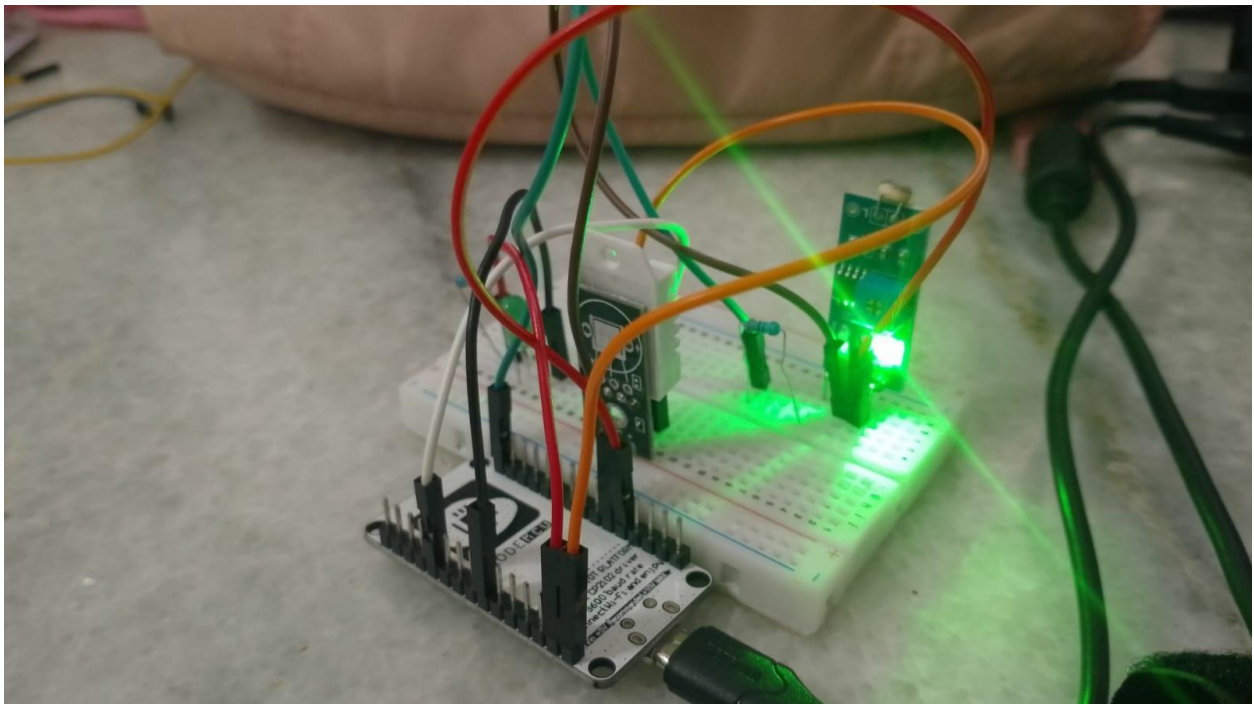
Kendali Relay

MATIKAN (OFF)

TUGAS



```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "salma";
6 const char* password = "0987654321";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18 <meta name="viewport" content="width=device-width, initial-scale=1">
19 <meta http-equiv="refresh" content="5">
20 <title>IoT Dashboard</title>
21 <style>
22 body { font-family: Arial; text-align: center; margin-top: 50px; }
23 button { padding: 15px 30px; font-size: 20px; border-radius: 4px; margin: 10px; cursor: pointer; }
24 .btn-on { background-color: #4CAF50; color: white; border: none; }
25 .btn-off { background-color: #F44336; color: white; border: none; }
26 .sensor-box { font-size: 24px; font-weight: bold; }
27 </style>
28 </head>
29 <body>
30 <h1>ESP8266 Web Server</h1>
31 <div class="sensor-box">
32 <p>Suhu Saat Ini: <strong>{TEMPERATURE}</strong> Celcius</p>
33 <p>Kelembapan: <strong>{HUMIDITY}</strong> %</p>
34 </div>
35 <h2>Kendali Relay</h2>
36 <button class="btn-on">
37 </body>
38 </html>
39 )rawliteral";
40
41 void handleRoot() {
42   String html = index_html;
43   float t = dht.readTemperature();
44   float h = dht.readHumidity();
45
46   if (!isnan(t)) {
47     html.replace("{TEMPERATURE}", "--");
48   }
49 }
```



Tujuan dari tugas ini adalah untuk memodifikasi struktur kerangka HTML mentah yang tersimpan di dalam memori NodeMCU agar mencapai interaktivitas tampilan otomatis (*Auto-Refresh*), sekaligus mengevaluasi limitasi dari arsitektur tradisional HTTP. Implementasi dilakukan pada sisi NodeMCU Server dengan menyisipkan perintah *Meta Refresh* HTML, yaitu `<meta http-equiv="refresh" content="5">` di dalam cakupan tag `<head>` pada variabel `index_html`. Perintah ini menginstruksikan *browser* untuk memuat ulang seluruh

halaman secara sekuensial setiap 5 detik sekali, sehingga nilai suhu dan kelembaban dari sensor DHT pada antarmuka web dapat diperbarui secara otomatis tanpa pengguna perlu menekan tombol *refresh* secara manual.